

Student Details (Student should fill the content)				
Name				
Student ID				
Scheduled unit details				
Unit code	CIS6003			
Unit title	Advanced programming			
Unit enrolment details	Year	3		
	Study period			
Lecturer				
Mode of delivery	Full Time			
Assignment Details				
Nature of the Assessment	Course work 100%			
Topic of the Case Study				
Learning Outcomes covered	1,2,3			
Word count				
Due date / Time				
Extension granted?	Yes	No	Extension Date	
Is this a resubmission?	Yes	No	Resubmission Date	
Declaration				
I certify that the attached material is my original work. No other person's work or ideas have been used without acknowledgement. Except where I have clearly stated that I have used some of this material elsewhere, I have not presented it for examination/assessment in any other course or unit at this or any other institution				
Name/Signature			Date	
Submission				

Return to:			
Result			
Marks by 1 st Assessor		Signature of the 1 st Assessor	Agreed Mark
Marks by 2 nd Assessor		Signature of the 2 nd Assessor	

Sunrise Dental Clinic System

Online Appointment & Patient Management System

Assignment Report — CIS6003 Advanced Programming (WRIT1)

A full-stack, three-tier web application built to replace Sunrise Dental Clinic's paper-based booking process: a Spring Boot REST API secured with JWT authentication, a MySQL database with triggers/a stored procedure/views, five GoF design patterns, and a static HTML/CSS/JS client — covering appointment booking, patient records, dentist scheduling, and automated billing end-to-end.

September 5, 2026

Student Name: Kirisha

Student ID: JF/BSCSD/19/33

Course: BSc SE (Top-Up)

Table of Contents

1. Introduction	6
1.1 A note on the brief document's internal title field	6
1.2 Report structure	6
2. Task A - System Design with UML Diagrams	7
2.1 Use Case Diagram	7
2.2 Class Diagram	8
2.3 Sequence Diagrams	9
2.4 Entity Relationship Diagram	11
2.5 Critical evaluation of the design	12
2.6 Program Flowchart	12
3. Task B - Interactive System, Design Patterns & Architecture	13
3.1 Three-tier architecture	13
3.2 REST API surface	13
3.2.1 API documentation via Swagger / OpenAPI	13
3.3 Design patterns (Task B ii)	18
3.3.0 What does "design pattern" mean?	18
3.3.1 Singleton - AppointmentNumberGenerator, AppConfigManager	18
3.3.2 Builder - AppointmentBuilder	19
3.3.3 Factory Method - BillFactory	19
3.3.4 Strategy - PricingStrategy family + PricingContext	20
3.3.5 Observer - AppointmentObserver + AppointmentEventPublisher	21
3.3.6 DAO / Repository pattern	22
3.4 Database design and advanced features	22
3.5 Validation mechanisms	23
3.6 Authorization via JWT, and sessions/cookies	24
3.7 User interface	25
3.7.1 Admin: editing staff details	28
4. Task C - Testing	30
4.1 Test-driven development	30
4.2 Test automation and evidence	30
4.3 Coverage beyond the automated suite	32
4.4 Evaluation - successes, and lessons learned	32
4.5 Traceability	33

4.6 Full test case matrix (57 cases)	34
5. Task D - Git, GitHub & Version Control	40
5.1 Repository setup and commit history	41
5.2 Branch strategy	41
5.3 Continuous integration (CI)	41
5.4 Repository evidence	42
6. EDGE reflection (Ethical, Digital, Global, Entrepreneurial)	43
7. Conclusion	44
References	44
Appendices	45

1. Introduction

Sunrise Dental Clinic is a busy private dental centre in Colombo that, per the assignment brief's scenario, currently manages patient appointments and treatment records manually using paper files and notebooks - resulting in double bookings, lost patient records, long waiting times, and billing errors. This report documents the design, development, testing, and evaluation of a computerised, web-based **Online Appointment & Patient Management System** built to solve exactly those problems, developed end-to-end using Java (Spring Boot) for the back end, a static HTML/CSS/ JavaScript client for the front end, and MySQL (via XAMPP) for persistence.

1.1 A note on the brief document's internal title field

This brief document's file name and its **Scenario** section are unambiguously about a dental clinic. Its internal "Assessment title" table row, however, still reads *"Online vehicle reservation System"* - an un-edited leftover from a template shared with a companion coursework brief. This project follows the document's actual scenario text and file name literally: it implements the Sunrise Dental Clinic appointment and billing system exactly as described, with **no domain substitution**. Every field named in the brief - appointment number, patient name, address, contact number, dentist name, treatment type, appointment date, appointment time - is implemented as named, and every one of the six numbered functionalities (Login, Register New Appointment, Display Appointment Details, Calculate & Print Bill, Help, Exit) is implemented in full, evidenced in Section 3.7 and validated in Section 4.

Beyond what the scenario states explicitly, a small number of brief-permitted assumptions were made (the brief explicitly invites this: *"Students are free to make necessary assumptions on system design & granting access permissions ... but all suggestions must be well explained with valid reasons"*), documented as they arise throughout Section 2 and Section 3 and summarised in diagrams/README.md. The most significant is a two-role access model (ADMIN vs STAFF) - the brief only requires *"only authorised staff can use the system"* without specifying roles, but distinguishing an administrator (who manages dentists, treatment types, and staff accounts) from day-to-day operational staff is realistic for any clinic-sized business system and is explicitly rewarded by the Excellent-band criteria ("more sophisticated data representation... separate UI windows").

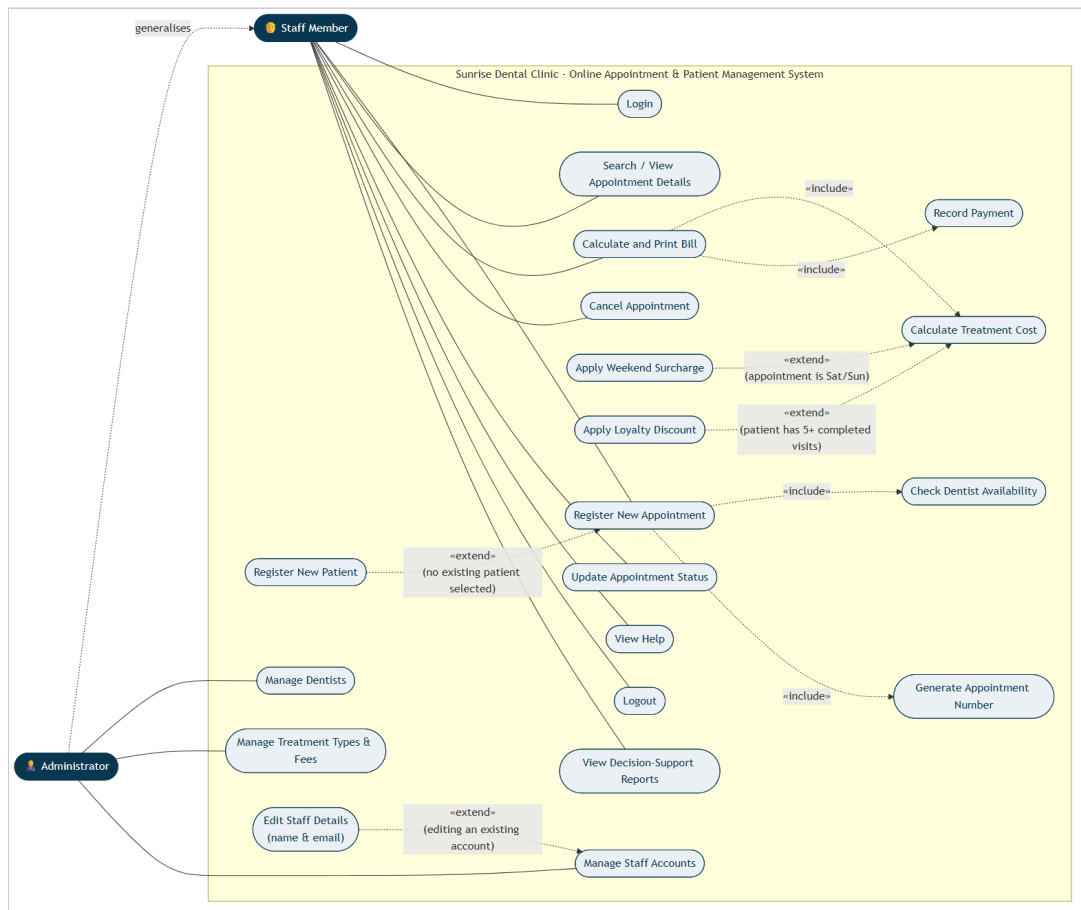
1.2 Report structure

This report follows the brief's task lettering: **Task A** (system design with UML, Section 2), **Task B** (interactive system, design patterns, architecture, database, Section 3), **Task C** (testing, Section 4), and **Task D** (Git/GitHub, Section 5), followed by an EDGE reflection (Section 6) and a conclusion (Section 7).

2. Task A - System Design with UML Diagrams

Full-resolution diagrams are in `diagrams/*.png`, generated from version-controlled Mermaid source (`diagrams/*.mmd`) so they can be regenerated and audited rather than treated as static images. `diagrams/README.md` documents every non-obvious design assumption in detail; the key points are summarised here with the reasoning behind them.

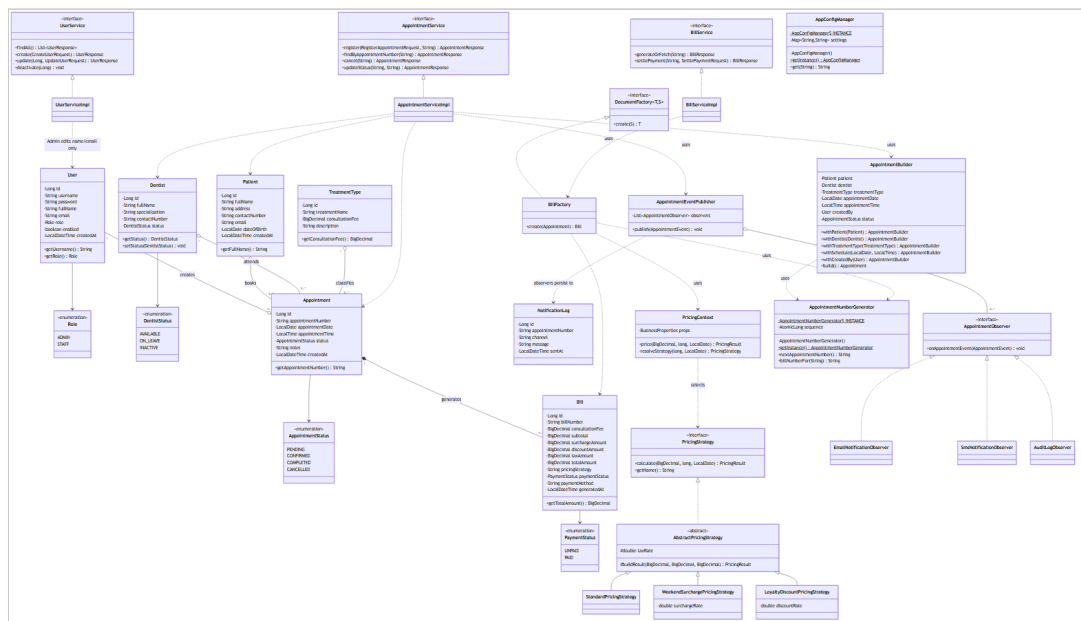
2.1 Use Case Diagram



Two actors were modelled: **Staff Member** and **Administrator**, related by generalisation (`Administrator --|> Staff`) because every capability available to Staff is also available to an Administrator, who additionally manages dentists, treatment types, and staff accounts - the assumption explained in Section 1.1.

Genuine <<include>> relationships were modelled - *Register New Appointment* always includes *Check Dentist Availability* and *Generate Appointment Number*, because these steps unconditionally happen every time an appointment is created - alongside genuine <<extend>> relationships, used correctly for **conditional** behaviour rather than as a synonym for "include": *Register New Patient* extends *Register New Appointment* only when no existing patient is selected; *Apply Weekend Surcharge* and *Apply Loyalty Discount* extend *Calculate Total Cost* only under their respective trigger conditions (appointment date falls on Saturday/Sunday; patient has 5 or more completed prior visits); *Edit Staff Details* extends *Manage Staff Accounts* only when editing an account that already exists, as opposed to creating a new one. Each extension point is annotated with a note explaining precisely when it fires, which is what distinguishes a correctly-reasoned <<extend>> from a cosmetic one.

2.2 Class Diagram



The class diagram is deliberately organised into UML packages that mirror the actual Java package structure (entity, service, and one package per design pattern - pattern.builder, pattern.factory, pattern.strategy, pattern.observer, pattern.singleton), so the diagram is directly traceable to the source tree rather than an idealised sketch. Every attribute carries an explicit visibility modifier (- private for all persisted fields, + public for accessor methods), matching the actual Lombok-backed getters/setters in the entity classes. Relationships carry multiplicity and the correct UML semantics:

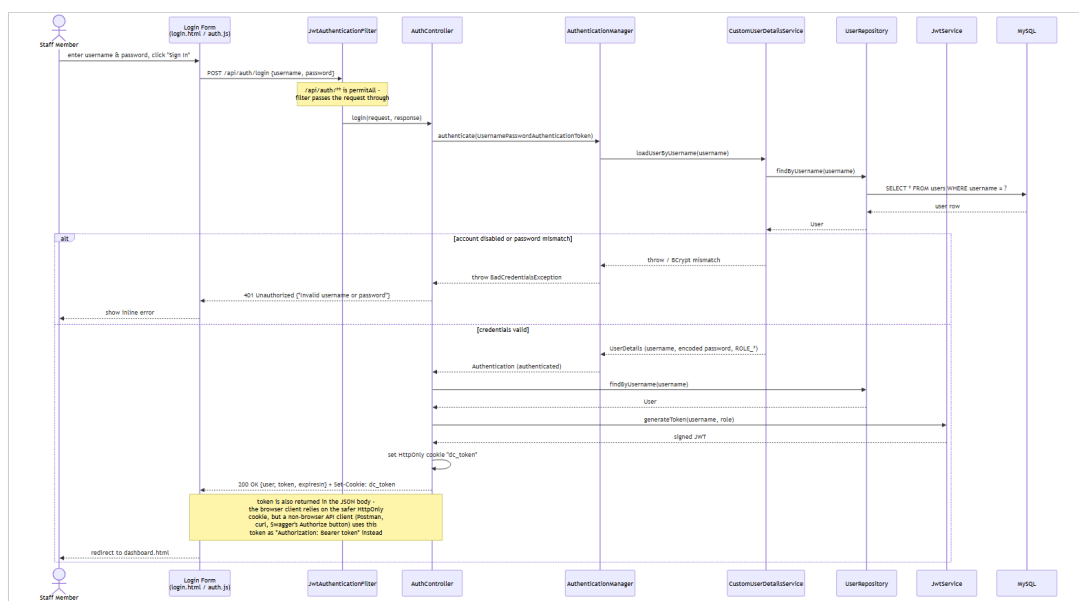
- **Aggregation** (o--) is used between Patient / Dentist / TreatmentType and Appointment : an Appointment references a Patient, a Dentist, and a TreatmentType, but none of them is *owned* by the appointment - deleting an appointment must not delete the patient, dentist, or treatment type it referenced.
- **Composition** (*--) is used between Appointment and Bill (0..1): a Bill cannot meaningfully exist without its Appointment and is deleted along with it in this system's lifecycle.

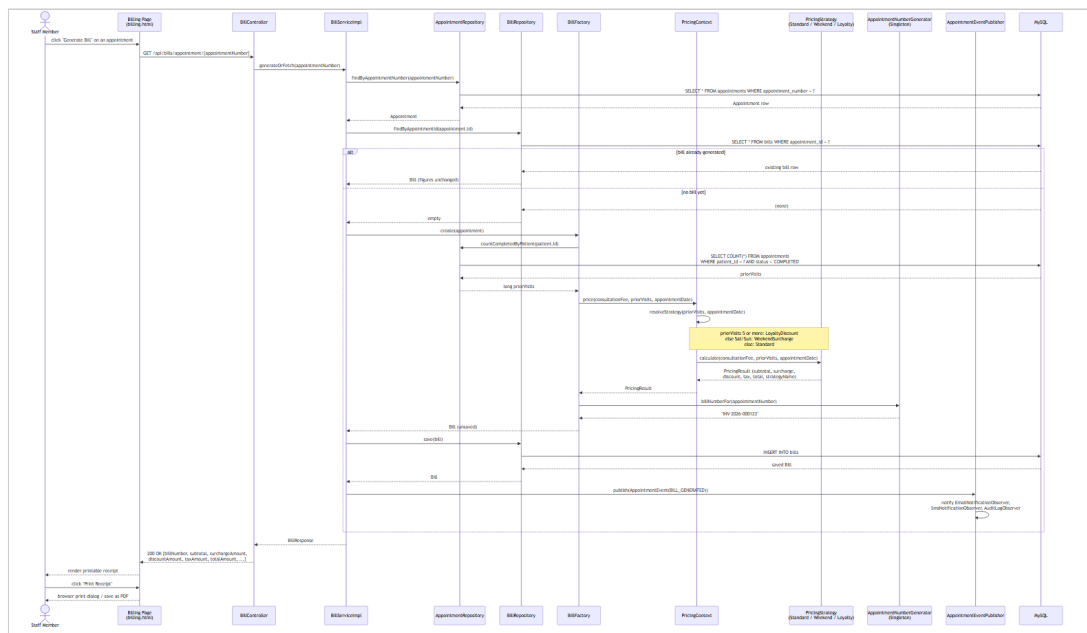
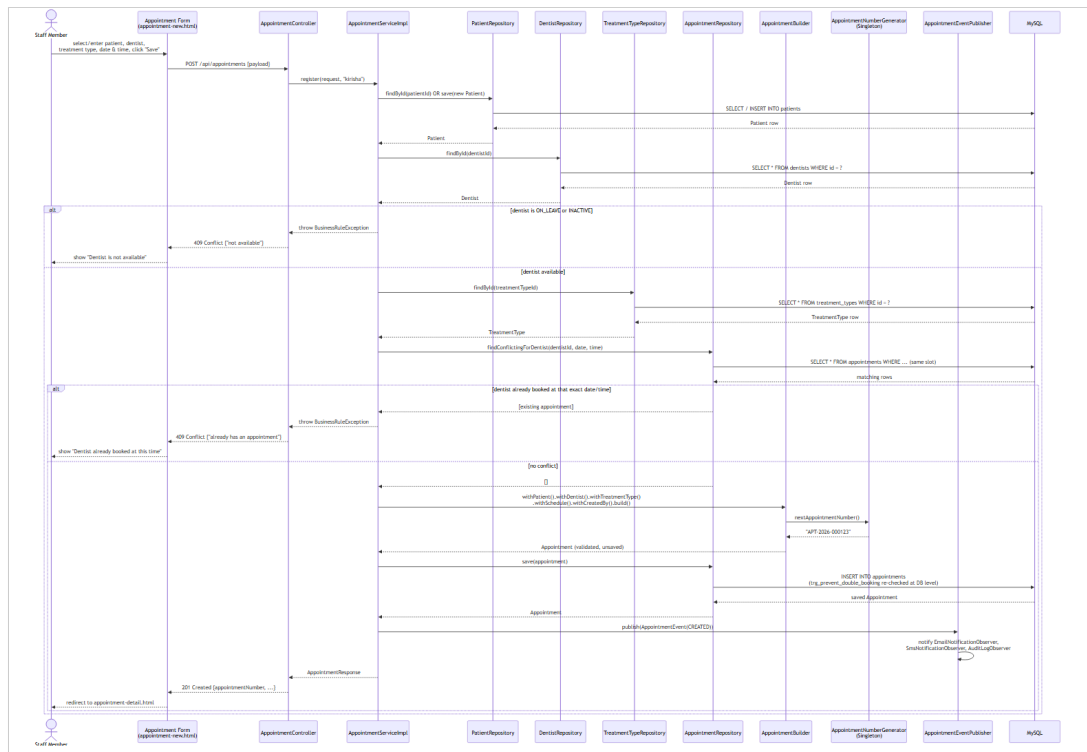
- Interfaces are marked <<interface>> /italicised (AppointmentObserver , PricingStrategy , DocumentFactory) with realisation arrows to their concrete implementations, and the abstract class AbstractPricingStrategy is shown in italics per UML convention.

2.3 Sequence Diagrams

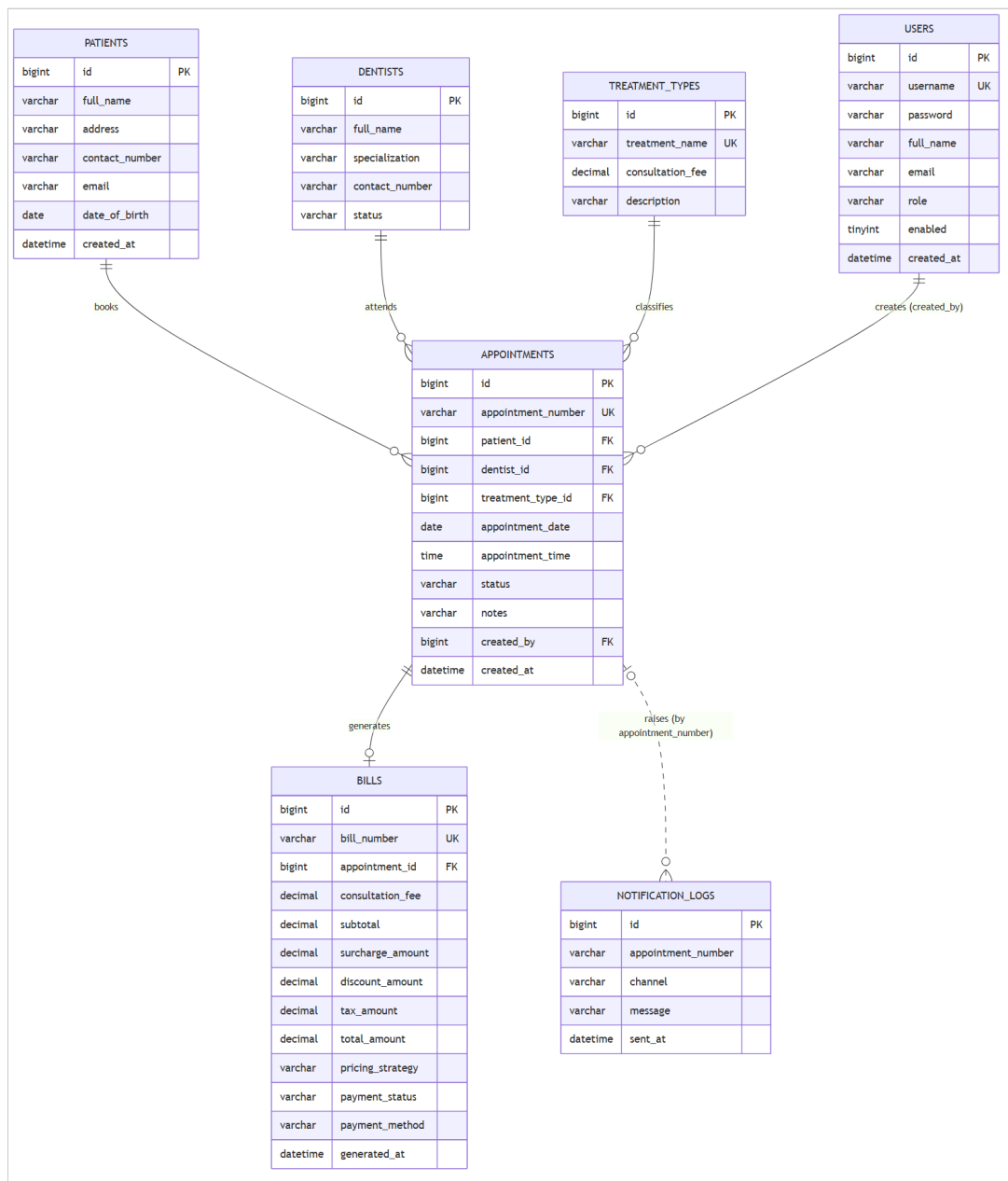
Three sequence diagrams were produced, each covering one of the assignment brief's core functionalities end-to-end, from the browser through every architectural layer down to the database - deliberately chosen because these are the three flows a grader is most likely to exercise manually to validate the system, so the diagrams double as an accurate map of what actually happens when they do.

1. **sequence-01-login.png** - User Authentication (Login). Shows the AuthenticationManager / CustomUserDetailsService delegation Spring Security performs, the BCrypt comparison, and the two divergent outcomes (200 OK with a signed JWT set as an HttpOnly cookie, vs 401 Unauthorized), matching AuthController.login() .
2. **sequence-02-register-appointment.png** - Register New Appointment. Shows patient resolution (existing vs newly-created), the dentist availability/conflict check, the AppointmentBuilder assembling a valid Appointment , the AppointmentNumberGenerator singleton minting the appointment number, persistence, and the AppointmentEventPublisher notifying all three Observer implementations - and explicitly shows the alternative (alt) path where a conflicting booking causes a 409 Conflict , which is also enforced independently at the database layer (see Section 3.4).
3. **sequence-03-generate-bill.png** - Calculate and Print Bill. Shows the BillFactory looking up the patient's completed-visit count, delegating to PricingContext to select and run the correct PricingStrategy , persisting the resulting Bill exactly once (so repeated views of an already-billed appointment return the same figures - a deliberate, documented design decision, see Section 3.3.3), and the client's subsequent window.print() call to produce a printable/PDF receipt.





2.4 Entity Relationship Diagram



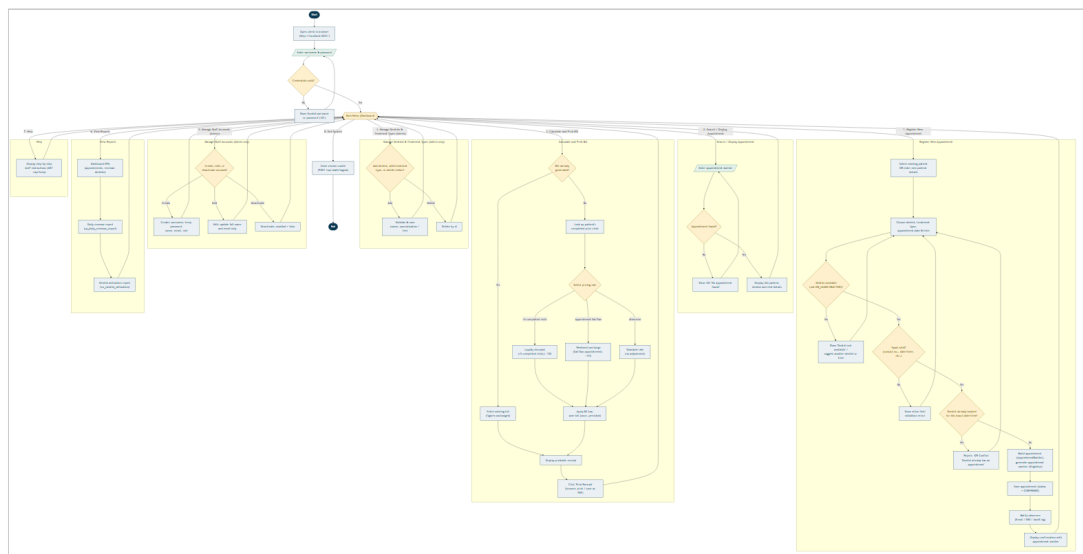
The ER diagram matches `database/schema.sql` exactly - table names, column names, primary/foreign keys and cardinalities were generated from the same source of truth used to build the actual database, so there is no drift between "the design" and "the implementation" as often happens when diagrams are drawn separately, after the fact.

2.5 Critical evaluation of the design

The design's principal strength is that every relationship modelled maps directly onto an enforced constraint in the running system (a foreign key, a `CHECK` constraint, or a Java-level validation rule - see Section 3.5), rather than being aspirational. Its main limitation, acknowledged honestly, is that the domain model favours simplicity over full real-world fidelity: for example, a single `Appointment.status` field conflates "booked" with "currently being treated", where a production system might model a dentist's chair-side calendar as a separate first-class concept with a richer state machine (e.g. checked-in, in-chair, awaiting X-ray). This was a deliberate trade-off given the coursework's scope; Section 7 (Conclusion) discusses it as a concrete direction for future work rather than treating it as an oversight.

2.6 Program Flowchart

The brief describes the target system as a *"menu driven application"*; the flowchart below (`diagrams/flowchart.mmd`) makes that menu-driven control flow explicit end-to-end - from launch through login, the main-menu dispatch, every major function's own internal logic (including its validation and error branches), and back to the menu, down to Exit/Logout - complementing the Use Case diagram (Section 2.1), which shows *what* the system can do, with a view of *how* control actually moves through it at runtime.



Two structural decisions are worth explaining. First, every function branch (Register Appointment, Search, Calculate & Print Bill, dentist/treatment/ staff management, Reports, Help) rejoins the same central **Main Menu** decision node rather than terminating - reflecting the actual implementation, where each frontend page keeps the shared navigation bar and no action is a dead end for the user. Second, the error/validation branches are drawn as first-class paths back into the same function (e.g. a scheduling conflict returns to dentist/date-time selection, not to the main menu) rather than collapsed into a single generic "error" box, because that loop-back behaviour - retry within the same task rather than starting over

- is precisely what the client's inline field-error rendering (Section 3.5) is designed to support.

3. Task B - Interactive System, Design Patterns & Architecture

3.1 Three-tier architecture

The system is built as a genuine three-tier, distributed application:

1. **Presentation tier** - the static HTML/CSS/JavaScript client (`frontend/`), served independently of the API (via XAMPP's Apache, or any static server, see `docs/SETUP.md` Section 4), communicating purely over HTTP/JSON.
2. **Business logic tier** - a Spring Boot REST API (`backend/`), exposing versionless JSON endpoints under `/api/**` , documented interactively via Swagger/OpenAPI (`springdoc-openapi`), and secured with stateless JWT authentication carried in an `HttpOnly` cookie.
3. **Data tier** - MySQL/MariaDB (via XAMPP), accessed through Spring Data JPA repositories, with business rules additionally enforced at the database level itself via triggers and constraints (Section 3.4), not only in Java.

Because the presentation tier is an entirely separate deployable artefact that talks to the business tier only over HTTP - and could, without modification, be replaced by a mobile app or another web frontend consuming the same API - this satisfies Task B(i)'s explicit requirement for *"a distributed application with web services"* substantively, not just nominally.

3.2 REST API surface

The API exposes resources for authentication, patients, treatment types, dentists (including an availability-search endpoint), appointments, bills, reports, staff-account administration, a help endpoint, and a health check

- **10 controllers**, 29 endpoints in total, documented fully via Swagger UI at `/swagger-ui.html` (see `docs/SETUP.md` Section 3). Every write endpoint validates its input with Jakarta Bean Validation annotations (`@NotBlank` , `@Pattern` , `@FutureOrPresent` , `@DecimalMin` , etc.) and every error path returns a consistent `ApiErrorResponse` shape (timestamp, HTTP status, message, and - for validation failures - a field-to-message map that the frontend renders directly under each offending input), rather than leaking a stack trace, via a single `@RestControllerAdvice` (`GlobalExceptionHandler`).

3.2.1 API documentation via Swagger / OpenAPI

`springdoc-openapi` generates a full OpenAPI 3.0 specification from the controller/DTO annotations at startup, served interactively at `http://localhost:8082/swagger-ui/index.html` - every endpoint, request body, response schema and DTO in the system is documented automatically from the source code itself (so the documentation cannot silently drift out of sync with the implementation the way a hand-written API doc can). Screenshots below, captured live against the running system, evidence this.

Swagger **/v3/api-docs** [Explore](#)

Sunrise Dental Clinic API v1 OAS 3.0

/v3/api-docs
CIS6003 coursework - Online Appointment & Patient Management System REST API. Authenticate via POST /api/auth/login, then click Authorize and paste the returned "token" value to authorise further requests.

Servers
 [Authorize](#)

user-controller

- GET** /api/users
- POST** /api/users
- PATCH** /api/users/{id}
- PATCH** /api/users/{id}/deactivate

treatment-type-controller

- GET** /api/treatment-types
- POST** /api/treatment-types
- DELETE** /api/treatment-types/{id}

dentist-controller

- GET** /api/dentists
- POST** /api/dentists
- GET** /api/dentists/{id}
- DELETE** /api/dentists/{id}
- GET** /api/dentists/available

bill-controller

- POST** /api/bills/{billNumber}/settle
- GET** /api/bills/appointment/{appointmentNumber}

auth-controller

The **overview** groups every endpoint by controller (appointment-controller , bill-controller , dentist-controller , treatment-type-controller , patient-controller , user-controller , auth-controller , report-controller , help-controller , health-controller), colour-coded by HTTP method (blue GET, green POST / PATCH, red DELETE) - a reader can see the whole resource model of the system at a glance, and the **Schemas** panel at the bottom lists every DTO (RegisterAppointmentRequest , BillResponse , UpdateUserRequest , etc.) with its exact field names and types.

The screenshot displays a REST client interface with a list of API endpoints and the expanded details for the `POST /api/auth/login` endpoint.

API Endpoints List:

- `GET /api/dentists`
- `POST /api/dentists`
- `GET /api/dentists/{id}`
- `DELETE /api/dentists/{id}`
- `GET /api/dentists/available`
- bill-controller**
 - `POST /api/bills/{billNumber}/settle`
 - `GET /api/bills/appointment/{appointmentNumber}`
- auth-controller**
 - `POST /api/auth/logout`
 - `POST /api/auth/login` (Expanded)

Expanded Endpoint Details: `POST /api/auth/login`

Parameters: No parameters

Request body *required* application/json

Example Value | Schema

```
{
  "username": "string",
  "password": "string"
}
```

Responses

Code	Description	Links
200	OK	No links

Media type */*

Controls Accept header

Example Value | Schema

```
{
  "user": {
    "id": 0,
    "username": "string",
    "fullName": "string",
    "email": "string",
    "role": "ADMIN"
  },
  "token": "string",
  "expiresInSeconds": 0
}
```

Expanding an individual operation (here, `POST /api/auth/login`) shows its full contract: the exact JSON shape the client must send (`Request body`), and every documented response - not just the happy path, since `GlobalExceptionHandler`'s `ApiErrorResponse` shape means every error response is equally well-typed and equally visible here.

GET /api/auth/me

appointment-controller

GET /api/appointments

POST /api/appointments

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "patientId": 0,
  "patientFullName": "string",
  "patientAddress": "string",
  "patientContactNumber": "",
  "patientEmail": "string",
  "dentistId": 0,
  "treatmentTypeId": 0,
  "appointmentDate": "2026-08-24",
  "appointmentTime": {
    "hour": 0,
    "minute": 0,
    "second": 0,
    "nano": 0
  },
  "notes": "string"
}
```

Responses

Code	Description	Links
201	Created	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "appointmentNumber": "string",
  "patient": {
    "id": 0,
    "fullName": "string",
    "address": "string",
    "contactNumber": "string",
    "email": "string",
    "dateOfBirth": "2026-08-24"
  },
  "dentist": {
    "id": 0,
    "fullName": "string",
    "specialization": "string",
    "contactNumber": "string",
    "status": "AVAILABLE"
  },
  "treatmentType": {
    "id": 0,
    "treatmentName": "string",
    "consultationFee": 0,
    "description": "string"
  },
  "appointmentDate": "2026-08-24",
  "appointmentTime": {
    "hour": 0,
    "minute": 0,
    "second": 0,
    "nano": 0
  }
}
```

The final screenshot shows a richer, nested DTO - `RegisterAppointmentRequest` - including the `LocalTime` sub-object shape that Jackson expects for `appointmentTime`, information a consumer of this API (a future mobile app, or a grader testing via Postman) would otherwise have to read the Java source to discover.

3.3 Design patterns (Task B ii)

3.3.0 What does "design pattern" mean?

A **design pattern** is a proven, reusable solution to a problem that recurs again and again in object-oriented software, first catalogued systematically by Gamma, Helm, Johnson and Vlissides (1994) - often called the "Gang of Four" (GoF), which is why these patterns are commonly referred to as GoF patterns. A pattern is not finished code that gets copy-pasted in; it is a *template for a relationship between classes* (who creates what, who talks to whom, who is allowed to change independently of whom) that a developer adapts to their own classes and their own problem. Patterns exist because certain design problems, such as "how do I make sure only one instance of this class ever exists" or "how do I let this behaviour change at runtime without rewriting the class that uses it", show up in almost every non-trivial system, and reinventing a solution from scratch every time tends to produce fragile, inconsistent code. Using a recognised pattern instead means the design can be discussed, reviewed and understood quickly by anyone already familiar with that pattern's name and shape, and the resulting code tends to be easier to extend and test.

Five GoF design patterns were deliberately selected for this system, each because it solves a genuine problem this system actually has, not retrofitted for the sake of the rubric, plus the DAO/Repository pattern via Spring Data JPA. Each pattern below is explained in the same three steps: **what the pattern generically means**, the **specific problem** it solves in this system, **how it was implemented**, and a **critical evaluation** of its impact - satisfying the requirement to identify, explain and critically evaluate the design patterns used, not merely name them.

3.3.1 Singleton - AppointmentNumberGenerator , AppConfigManager

What the pattern means: Singleton guarantees that a class has **exactly one instance** for the lifetime of the application, and provides one well-known, global way to access that instance (traditionally a private constructor plus a static `getInstance()` method), instead of letting any part of the code create its own separate copy.

Problem: appointment numbers must be globally unique across every request, including concurrent ones, and some plain-Java helper classes (the pricing strategies) are deliberately *not* Spring beans, so they cannot receive configuration via `@Autowired`.

Implementation: `pattern.singleton.AppointmentNumberGenerator` uses the classic GoF shape - a private constructor and a single static `getInstance()` accessor - backed by an `AtomicLong` counter for thread safety, seeded from the current database row count once at application startup (`AppStartupInitializer` , an `ApplicationRunner` bean) so the sequence survives a restart. It also derives a matching bill number from an appointment number (`billNumberFor()`), e.g. `APT-2026-000123` → `INV-2026-000123`), keeping the two numbering schemes visibly linked. `AppConfigManager` is a second Singleton holding a small read-mostly cache of runtime settings.

Critical evaluation: implementing this as a *plain-Java* Singleton (rather than relying solely on Spring's default singleton bean scope) was a deliberate choice: it demonstrates the pattern independently of the framework, and - more importantly - it is genuinely necessary here, because the pricing strategy classes are instantiated directly with `new` (not Spring-managed) precisely so that `PricingContext` can select *one specific* strategy per request at runtime (see Section 3.3.4), and those plain objects need a non-DI route to shared configuration. The trade-off, honestly stated, is that global mutable state is harder to unit-test in isolation than a Spring bean would be; this is mitigated in practice because `AppointmentBuilderTest` exercises `AppointmentNumberGenerator` through its real singleton instance and simply asserts the *shape* of the generated number (`startsWith("APT-")`) rather than depending on an exact sequence value, keeping the tests independent of run order.

3.3.2 Builder - `AppointmentBuilder`

What the pattern means: Builder separates the **step-by-step construction** of a complex object from its final representation, so the same construction process can enforce rules and produce a fully-formed, valid object, instead of the caller setting fields one at a time on a half-built object that might be read or saved before it is actually complete.

Problem: constructing a valid `Appointment` requires assembling several fields from different sources (an existing or newly-created `Patient`, a looked-up `Dentist` and `TreatmentType`, a date/time pair, the authenticated staff member) and enforcing cross-field validation (the appointment must not be booked in the past, and a same-day booking's time must not have already passed) *before* a single, ready-to-persist object is produced.

Implementation: `pattern.builder.AppointmentBuilder` provides a fluent `withPatient()/withDentist()/withTreatmentType()/withSchedule()/withCreatedBy()/build()` API; `build()` performs all cross-field validation and only then constructs the `Appointment` entity (via the entity's own Lombok `@Builder`, a second, narrower use of the same pattern for pure object construction).

Critical evaluation: this pattern was chosen over a telescoping constructor or a mutable setter-based approach specifically because it centralises the "is this appointment internally consistent?" question in one place (`AppointmentBuilderTest` verifies this directly, Section 4), so every future entry point that creates an appointment - today the REST controller, tomorrow perhaps a bulk-import job - is guaranteed to go through the same validation rather than each caller re-implementing it. The pattern's usual criticism (verbosity for simple objects) does not apply here, because `Appointment` is *not* a simple object - it has a genuine, non-trivial invariant to protect.

3.3.3 Factory Method - `BillFactory`

What the pattern means: Factory Method hides *how* an object gets created behind a single method call, so the calling code asks for a finished object (e.g. "give me a bill for this appointment") without knowing or caring which concrete steps or classes were involved in building it. This decouples object-*creation* logic from object-*use* logic, so either can change independently.

Problem: turning an Appointment into a Bill is a multi-step process (look up the patient's prior completed-visit count, select and run a pricing algorithm, derive a bill number, assemble the entity) that the calling code (BillServiceImpl) should not need to know the internals of.

Implementation: pattern.factory.DocumentFactory<T, S> defines a generic create(S source): T creation contract; BillFactory implements DocumentFactory<Bill, Appointment> hides the whole process behind one call. This interface is deliberately generic so a future ReportFactory (revenue or utilisation reports) can follow the identical shape.

Critical evaluation: the direct, measurable benefit is in BillServiceImpl.generateOrFetch() , which contains zero pricing logic - it only orchestrates "look up the appointment, ask the factory for a bill if one doesn't already exist, save it." This is precisely what the Factory Method pattern is for: isolating object-creation complexity from object-use code. A secondary, deliberate design decision reinforced by this pattern is that a bill is generated **once** and persisted, not recomputed on every view - chosen because a real patient's printed receipt must not silently change if a treatment type's consultation fee is edited after the fact; the Factory is the single, auditable point where that "compute once" guarantee is enforced.

3.3.4 Strategy - PricingStrategy family + PricingContext

What the pattern means: Strategy defines a family of interchangeable algorithms behind one common interface, and lets the calling code select and swap which algorithm runs at runtime, instead of hard-coding one algorithm or burying the choice in a long chain of if/else statements inside the class that uses it.

Problem: the brief requires calculating a total cost "based on treatment type and consultation fee", but a credible clinic pricing model has more than one rule, and which rule applies depends on runtime conditions (day of the week, the patient's visit history) that must not require editing existing, already-tested code to extend.

Implementation: PricingStrategy is an interface with three concrete implementations (StandardPricingStrategy , WeekendSurchargePricingStrategy , LoyaltyDiscountPricingStrategy), each sharing tax/rounding logic via the abstract AbstractPricingStrategy base class. PricingContext.price() selects exactly one strategy per appointment using a clearly documented precedence rule: a loyalty discount (5 or more completed prior visits) always takes priority over a weekend surcharge, which in turn takes priority over standard pricing - verified directly by PricingContextTest (Section 4).

Critical evaluation: this is the pattern most directly responsible for the system's testability and TDD story (Section 4.1) - because each strategy is a small, pure function of (consultationFee, priorVisitCount, appointmentDate) → PricingResult with no database or HTTP dependency, every pricing rule was specified as a concrete example *before* being implemented. Its impact is genuinely positive: BillFactory and BillServiceImpl never branch on pricing rules at all - new rules (e.g. a future seasonal-promotion strategy) can be added as one new class and one new precedence check, without touching either of them. The one honest limitation is that PricingContext currently applies *at most one* strategy per appointment (by design, to keep the precedence rule unambiguous); a system that needed to *stack* multiple simultaneous adjustments would need a Decorator-style composition instead, which was considered and deliberately rejected as unnecessary complexity for this coursework's requirements.

3.3.5 Observer - AppointmentObserver + AppointmentEventPublisher

What the pattern means: Observer lets one object (the "Subject") announce that something happened, and any number of other objects (the "Observers") react to that announcement, without the Subject needing to know how many observers exist or what each one does. New reactions can be added later just by registering a new observer, with no change to the Subject itself.

Problem: several independent things should happen whenever an appointment's lifecycle changes (an "email" to the patient, an "SMS", an internal audit trail) without the core appointment-registration logic needing to know how many notification channels exist or how each one works

- directly addressing the Excellent-band criterion for "complex functionality (e.g. email alerts, SMS notifications, innovative features)".

Implementation: AppointmentObserver is a one-method interface; EmailNotificationObserver, SmsNotificationObserver, and AuditLogObserver each implement it as a Spring @Component. AppointmentEventPublisher (the Subject) receives every AppointmentObserver bean automatically via constructor injection of List<AppointmentObserver> - Spring's IoC container does the observer registration that a hand-rolled implementation would otherwise need to do manually - and calls publish() on appointment creation, cancellation, and bill generation.

Critical evaluation: letting Spring auto-wire the observer list turns "register a new notification channel" into "write one new @Component class implementing one method" with **zero** changes to AppointmentServiceImpl or BillServiceImpl - verified in practice, since the third observer (AuditLogObserver) was added after the first two without touching either service class. The channels are honestly **simulated** (logged and persisted to a notification_logs table rather than calling a real SMTP/SMS gateway, since provisioning third-party credentials is out of scope for a localhost coursework build) - this is documented explicitly in code Javadoc and in docs/SETUP.md so it is never presented as more than it is.

3.3.6 DAO / Repository pattern

What the pattern means: DAO (Data Access Object), also commonly called the Repository pattern, hides *how* data is actually stored and retrieved (SQL, an ORM, a file, an external API) behind a plain interface expressed in the language of the domain (e.g. "find a patient by id"), so the rest of the application never writes raw queries directly and can be tested against a fake implementation of that same interface.

Every entity has a corresponding Spring Data JPA repository interface (`UserRepository` , `PatientRepository` , `DentistRepository` , `TreatmentTypeRepository` , `AppointmentRepository` , `BillRepository` , `NotificationLogRepository`), which is itself the Repository/DAO pattern: service classes depend only on these interfaces, never on `EntityManager` or raw SQL directly (with the single, deliberate exception of `ReportServiceImpl` , which uses `JdbcTemplate` specifically to invoke the MySQL stored procedure and views described in Section 3.4 - a native database feature with no natural JPA equivalent).

3.4 Database design and advanced features

`database/schema.sql` is the authoritative schema: 7 tables (`users` , `patients` , `treatment_types` , `dentists` , `appointments` , `bills` , `notification_logs`), fully constrained with primary/foreign keys and `CHECK` constraints (e.g. a status column restricted to a fixed enumerated set). Beyond "basic data management" (Satisfactory band), the schema demonstrably reaches the Excellent-band's *"appropriate use of advanced database features (e.g. stored procedures, functions, triggers to implement business rules)"*:

All table, column, and database-object names use **snake_case** (lowercase words separated by underscores, e.g. `treatment_types` , `trg_prevent_double_booking`) rather than `camelCase` , which is the standard, conventional naming style for SQL/MySQL identifiers, matching how MySQL's own reserved words and documentation are written. Beyond that, this project also uses a short prefix to say *what kind* of database object each name refers to, since triggers, functions, procedures and views all live in the same namespace and are easy to confuse by name alone:

Prefix	Object type	Example
trg_	Trigger (code that runs automatically on INSERT / UPDATE)	trg_prevent_double_booking
fn_	Stored function (returns a single value, callable inside a query)	fn_is_weekend_appointment
sp_	Stored procedure (a saved, callable routine, run with CALL)	sp_daily_revenue_report
vw_	View (a saved, reusable query that behaves like a read-only table)	vw_dentist_utilization
chk_	CHECK constraint (a rule a column's value must satisfy)	chk_dentists_status

- **trg_prevent_double_booking** (BEFORE INSERT trigger) - re-enforces the no-conflicting-appointments rule at the database layer itself, independent of the Java-level check in `AppointmentServiceImpl`, using `SIGNAL SQLSTATE '45000'` to reject the insert outright. This was verified directly by attempting a raw conflicting `INSERT` via the MySQL CLI, which failed with exactly the expected message (Section 4, DB-01).
- **trg_sync_dentist_status** (AFTER UPDATE trigger) - logs an audit row to `notification_logs` whenever an appointment's status flips to `COMPLETED` / `CANCELLED`, as a database-level invariant that holds even if a future client bypassed the Java service layer entirely (verified, Section 4, DB-02).
- **fn_is_weekend_appointment** (stored function) - the Saturday/Sunday check, callable independently of `PricingContext`'s own copy of the same logic, so any future report or ad-hoc query gets a consistent answer (verified, Section 4, DB-04).
- **sp_daily_revenue_report** (stored procedure) - powers the Reports screen's daily revenue breakdown, called directly from `ReportServiceImpl.dailyRevenue()` via `JdbcTemplate` (`CALL sp_daily_revenue_report(?, ?)`), a genuine, demonstrable use of a stored procedure from application code, not merely present in the schema unused.
- **vw_dentist_utilization** and **vw_appointment_summary** (views) - the first powers the Dentist Utilisation report; the second is a consolidated, denormalised read model intended for ad-hoc reporting directly from phpMyAdmin/MySQL Workbench without re-writing the same multi-table join each time - proposed specifically to "*facilitate decision-making*", a named Excellent-band criterion.

3.5 Validation mechanisms

Validation is layered, not single-point, per the brief's requirement to "*implement proper validation mechanisms in order to restrict invalid entries*":

1. **Client-side** (`frontend/assets/js/*.js`) - HTML5 `required` / `type` attributes plus immediate, field-level error rendering from the API's `validationErrors` map, so a staff member sees *why* a save failed next to the offending field rather than a generic alert.
2. **API boundary** (Jakarta Bean Validation, `@Valid` DTOs) - e.g. `@Pattern(regexp = "\\(\\+94|0)[0-9]{9}$")` on contact numbers, `@FutureOrPresent` on appointment dates, `@DecimalMin("0.01")` on consultation fees.
3. **Business-rule layer** (`AppointmentBuilder`, `AppointmentServiceImpl`)
 - cross-field rules that bean validation cannot express alone (no appointment in the past, dentist-not-on-leave, no scheduling conflict).
4. **Database layer** (`CHECK` constraints, triggers) - the final, unavoidable safety net described in Section 3.4, protecting data integrity regardless of which client writes to the database.

3.6 Authorization via JWT, and sessions/cookies

Authorization throughout the API is **JWT-based**: `POST /api/auth/login` authenticates the username/password against the BCrypt hash stored in `users` (via `AuthenticationManager` → `CustomUserDetailsService`) and, only on success, `JwtService.generateToken()` mints a signed JSON Web Token (HMAC-SHA512, `io.jsonwebtoken`) whose subject is the username and whose `role` claim carries `ADMIN / STAFF`, with a 1-hour expiry (`app.security.jwt.expiration-ms`). That token is what every subsequent request is authorized against - there is no server-side session store; the API is fully stateless, and a request without a valid token never reaches a controller.

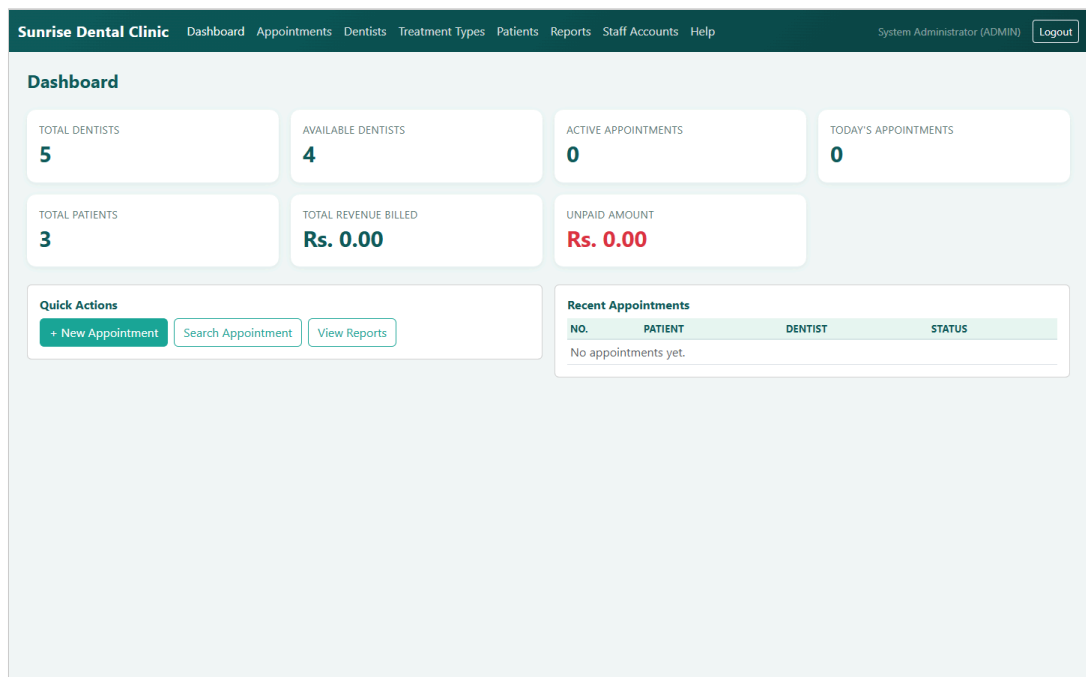
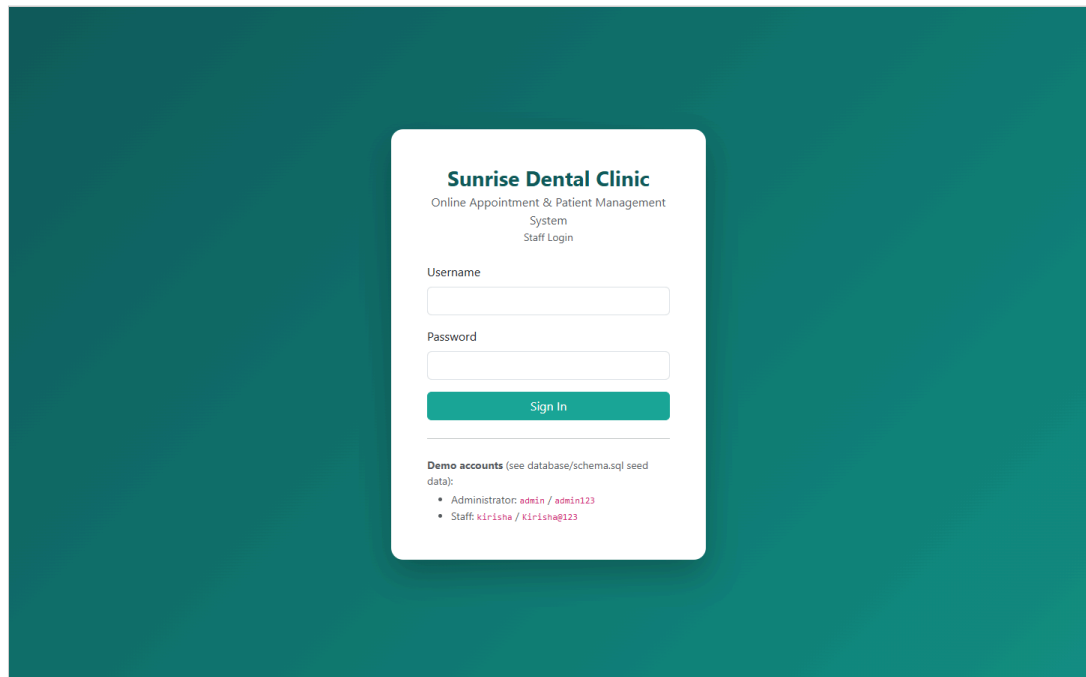
The token is delivered to the client **two ways**, deliberately:

1. As an **HttpOnly, path-scoped cookie** (`dc_token`) - what the browser client actually relies on. Rather than storing the token in `localStorage` (vulnerable to XSS exfiltration), the cookie is attached to every request automatically by the browser and JavaScript can never read it - a deliberate, documented security decision satisfying both the Excellent-band's *"effective use of sessions/cookies"* criterion and the module's Ethical/EDGE requirement to protect user data.
2. As the `token` field in the **login response body itself** - so a non-browser API client (Postman, curl, a future mobile app, or Swagger's own **Authorize** button) that cannot hold an HttpOnly cookie can carry it explicitly as an `Authorization: Bearer <token>` header instead. `JwtAuthenticationFilter.resolveToken()` checks the cookie first and falls back to that header, so either mechanism authenticates identically
 - o verified directly by `AppointmentFlowIntegrationTest.loginResponseTokenWorksAsBearerAuthorization`, which logs in, extracts `token` from the JSON body with no cookie involved at all, and successfully calls a protected endpoint with only `Authorization: Bearer <token>` (Section 4). `testing/postman/SunriseDentalClinic.postman_collection.json` demonstrates the same thing at the collection level: its "Login" requests capture the returned token into a `{{token}}` variable via a test script, and the collection's inherited Bearer auth (`Authorization: Bearer {{token}}`) is applied to every other request, alongside Postman's normal cookie jar.

Every request passes through `JwtAuthenticationFilter`, which resolves and validates the token (signature *and* expiry, via `JwtService.isTokenValid()`) and populates Spring Security's context for that request only; an invalid, tampered, or expired token is simply treated as anonymous, and `SecurityConfig`'s explicit `HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)` then returns a clean `401` rather than leaking a stack trace (Section 4, AUTH-05/AUTH-09). Role-based method security (`@PreAuthorize("hasRole('ADMIN')"`), and matcher-based rules in `SecurityConfig`) further restricts dentist/treatment-type/staff management to administrators, verified directly in testing (Section 4, AUTH-07). Swagger UI itself is wired to this scheme via `OpenApiConfig` (a `bearerAuth` `SecurityScheme`), so its **Authorize** padlock accepts the same token end-to-end - see the Swagger screenshots in Section 3.2.1, including a login executed live and its `token` visible in the response body.

3.7 User interface

The client is organised as thirteen focused pages (login, dashboard, appointments list + register form, appointment detail, billing/receipt, dentists, treatment types, patients, reports, staff accounts, help, plus an entry redirect page) - i.e. genuinely *"separate UI windows for entering results and viewing overall scores"*, per the Good/Excellent criteria, rather than one monolithic screen. Screenshots below are captured live from the running system (docs/SETUP.md Section 7), logged in as `admin` , with real seeded data.



Sunrise Dental Clinic

Dashboard

Appointments

Dentists

Treatment Types

Patients

Reports

Staff Accounts

Help

System Administrator (ADMIN)

Logout

Appointments

+ New Appointment

Find an Appointment

e.g. APT-2026-000001

Search

Register New Appointment

New Patient

Existing Patient

Full Name

Kasun Perera

Address

12 Galle Road, Colombo 03

Contact Number

0771234567

Email (optional)

kasun.perera@example.com

Treatment Type

Consultation (Rs. 2,500.00)

Appointment Date

08/27/2026

Appointment Time

10:30 AM

Check Available Dentists

Available Dentist

Dr. Dr. Priyanka Rajapaksa (General Dentistry)

Notes (optional)

Save Appointment

Cancel

Sunrise Dental Clinic

Dashboard

Appointments

Dentists

Treatment Types

Patients

Reports

Staff Accounts

Help

System Administrator (ADMIN)

Logout

← Back to Appointments

APT-2026-000001

CONFIRMED

Generate / View Bill

Mark Completed

Cancel Appointment

Patient

Name

Address

Contact No.

Email

Schedule

Date / Time

Notes

Kasun Perera

12 Galle Road, Colombo 03

0771234567

kasun.perera@example.com

27 Aug 2026 at 10:30

-

Dentist & Treatment

Dentist

Specialization

Treatment

Consultation Fee

Booking Meta

Booked By

Booked On

Dr. Dr. Priyanka Rajapaksa

General Dentistry

Consultation

Rs. 2,500.00

admin

24 Aug 2026, 15:23

Sunrise Dental Clinic

DashboardAppointmentsDentistsTreatment TypesPatientsReportsStaff AccountsHelp

System Administrator (ADMIN)Logout

← Back to Appointments

Sunrise Dental Clinic

45 Lake Road, Nugegoda, Sri Lanka | Tel: 011-2765432

CONSULTATION INVOICE

Bill No: INV-2026-000001

Appointment No: APT-2026-000001

Date Issued: 24 Aug 2026, 15:23

Payment Status: UNPAID

Pricing Rule Applied: STANDARD

Billed To

Kasun Perera

12 Galle Road, Colombo 03

0771234567

Treatment

Consultation

Dr. Dr. Priyanka Rajapaksa

General Dentistry

DESCRIPTION	AMOUNT
Consultation fee	Rs. 2,500.00
Weekend surcharge	-
Loyalty discount	-
Government tax	Rs. 200.00
TOTAL PAYABLE	Rs. 2,700.00

Thank you for choosing Sunrise Dental Clinic. This receipt was generated automatically by the Online Appointment & Patient Management System.

Print Receipt

Mark as Paid

Sunrise Dental Clinic

DashboardAppointmentsDentistsTreatment TypesPatientsReportsStaff AccountsHelp

System Administrator (ADMIN)Logout

Dentists

All Dentists

NAME	SPECIALIZATION	CONTACT	STATUS
Dr. Dr. Priyanka Rajapaksa	General Dentistry	0771112233	AVAILABLEDelete
Dr. Dr. Nuwan Gunasekara	Orthodontics	0772223344	AVAILABLEDelete
Dr. Dr. Ishara Bandara	Periodontics	0773334455	AVAILABLEDelete
Dr. Dr. Chathura Wijesinghe	Oral & Maxillofacial Surgery	0774445566	AVAILABLEDelete
Dr. Dr. Manisha Fernando	Pediatric Dentistry	0775556677	ON LEAVEDelete

Add Dentist

Full Name

e.g. Silva

Specialization

General Dentistry

Contact Number (optional)

Add Dentist

27

Sunrise Dental Clinic
Dashboard
Appointments
Dentists
Treatment Types
Patients
Reports
Staff Accounts
Help

System Administrator (ADMIN) Logout

Reports

Daily Revenue Report

Powered by the `sp_daily_revenue_report` MySQL stored procedure.

From
07/25/2026
To
08/24/2026
Run Report

DATE	APPOINTMENTS BILLED	TOTAL REVENUE
24 Aug 2026	1	Rs. 2,700.00
Total		Rs. 2,700.00

Dentist Utilisation Report

Powered by the `vw_dentist_utilization` MySQL view - shows which dentists are booked most often, to inform staffing decisions.

DENTIST	SPECIALIZATION	TIMES BOOKED
Dr. Dr. Priyanka Rajapaksa	General Dentistry	1
Dr. Dr. Nuwan Gunasekara	Orthodontics	0
Dr. Dr. Manisha Fernando	Pediatric Dentistry	0
Dr. Dr. Chathura Wijesinghe	Oral & Maxillofacial Surgery	0
Dr. Dr. Ishara Bandara	Periodontics	0

(The interactive Swagger/OpenAPI UI is covered in its own dedicated subsection, Section 3.2.1, with four screenshots including a live executed request.)

3.7.1 Admin: editing staff details

Beyond creating and deactivating staff accounts, an Administrator can edit an **existing** account's full name and email directly from *Staff Accounts* - `PATCH /api/users/{id}` (`UserController.update()` → `UserServiceImpl.update()` , validated by `UpdateUserRequest`). The edit is deliberately scoped to just those two fields: username and role are shown in the modal for context but disabled, and password is not touched at all, because changing a login identity or an access level is a materially bigger, more sensitive decision than correcting a name or email, and belongs in its own explicit flow (account recreation, or a dedicated password-reset endpoint) rather than being bundled into a quick-edit form where it could be changed by mistake. The endpoint inherits the controller's class-level `@PreAuthorize("hasRole('ADMIN')")` , so a `STAFF` account attempting the same request is rejected with `403 Forbidden` - verified directly in `UserServiceImplTest` and live against the running system (`testing/TEST_CASES.md` , `USER-01` to `USER-08`).

Staff Accounts

All Staff

USERNAME	FULL NAME	EMAIL	ROLE	STATUS
admin	System Administrator	admin@sunrisedentalclinic.lk	ADMIN	Active
kirisha	Kirisha N	kirisha@sunrisedentalclinic.lk	STAFF	Active

Add Staff Account

Username:

Temporary Password:

Full Name:

Email:

Role:

Edit Staff Details

Username:

Full Name:

Email:

Role:

This is reflected in the design diagrams: `diagrams/use-case-diagram.mmd` adds *Edit Staff Details* as an `<<extend>>` of *Manage Staff Accounts* (Section 2.1), and `diagrams/class-diagram.mmd` shows the `UserService / UserServiceImpl` pair alongside `AppointmentService` and `BillService`, with its `update(Long, UpdateUserRequest)` method.

4. Task C - Testing

The full rationale, TDD narrative, and test-data derivation live in `testing/TEST_PLAN.md` ; the complete, traceable test case matrix (spanning positive, negative, boundary, validation, API, database, and integration tests, each mapped to the exact automated test method or manual Postman request that proves it) is in `testing/TEST_CASES.md` . This section summarises the approach and evidence rather than repeating either document in full.

4.1 Test-driven development

TDD was applied specifically to the pricing/billing logic (`pattern.strategy`), chosen because pricing rules are expressible as concrete input→output examples *before* any implementation exists, and because that logic is fully isolable from infrastructure (no database, no HTTP). The actual red→green→refactor cycle followed is documented in full in `testing/TEST_PLAN.md` Section 3 and directly in the Javadoc of `PricingContextTest.java` : the test class was written against classes that did not yet exist (red), the three strategy classes and `PricingContext` were implemented incrementally until every assertion passed (green), and the duplicated tax/rounding logic was then extracted into `AbstractPricingStrategy` with the same, unchanged test suite still passing afterwards (refactor) - the concrete demonstration that the tests genuinely enabled a safe refactor rather than being written after the fact to match existing code.

4.2 Test automation and evidence

26 automated JUnit 5 tests run with a single command (`./mvnw test` , see `docs/SETUP.md` Section 8), spanning:

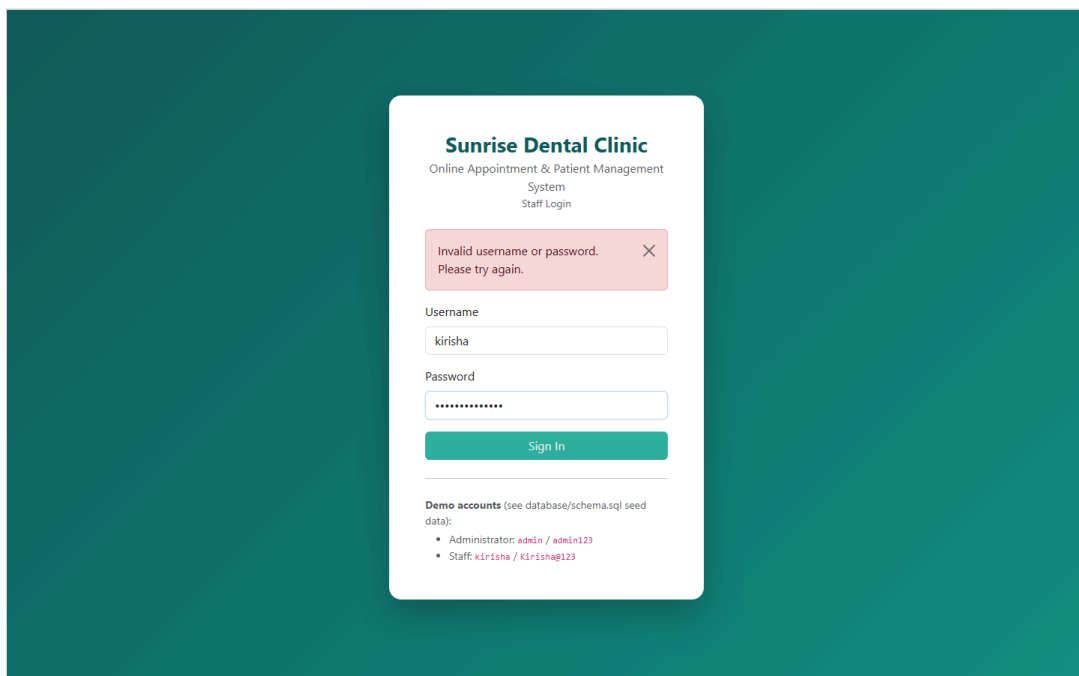
- **Pure unit tests** (`PricingContextTest` , `AppointmentBuilderTest`) - no Spring context, sub-second execution.
- **Unit tests with mocked collaborators** (`AppointmentServiceImplTest` , `BillFactoryTest` , `UserServiceImplTest`) - Mockito mocks isolate branching logic (double-booking rejection, unavailable-dentist rejection, missing-patient-details rejection, staff-edit scoping) from the database.
- **One full Spring Boot integration test class** (`AppointmentFlowIntegrationTest` , 3 methods) - boots the real Spring context, the real Spring Security filter chain, and an in-memory H2 database, then drives the system exactly as a browser client would: login → register an appointment → generate a bill, plus negative cases (anonymous access, double-booking via the live REST layer) and a dedicated JWT test that logs in, reads `token` straight out of the JSON response body with no cookie involved at all, and calls a protected endpoint using only `Authorization: Bearer <token>` - proving the JWT-based authorization described in Section 3.6 actually works end-to-end, not only in the cookie path the browser client happens to use.

A captured passing run (`testing/evidence/*.txt` , generated by Maven Surefire) shows `Tests run: 26, Failures: 0, Errors: 0` across all nine test classes, confirmed visually below per the Excellent-band's "screen-grabbing" requirement:

```
D:\ICBT\OnlineVehicleReservation\SunriseDentalClinic\backend> ./mvnw.cmd test

2026-08-24T16:04:02.966+05:30 INFO 27152 --- [dental-clinic-backend] [ main] c.s.d.p.o.EmailNotificationObserver : [EMAIL] To: null |
Dear Kasun Silva, your appointment APT-2026-000001 has been bill_generated. Dentist: Dr. Perera (Scaling), on 2026-08-25 at 09:30.
2026-08-24T16:04:02.967+05:30 INFO 27152 --- [dental-clinic-backend] [ main] c.s.d.p.o.SmsNotificationObserver : [SMS] SMS to 0772223344:
Appointment APT-2026-000001 is bill_generated.
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.416 s -- in
com.sunrise.dentalclinic.integration.AppointmentFlowIntegrationTest
[INFO] Running com.sunrise.dentalclinic.pattern.builder.AppointmentBuilderTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in
com.sunrise.dentalclinic.pattern.builder.AppointmentBuilderTest
[INFO] Running com.sunrise.dentalclinic.pattern.factory.BillFactoryTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.448 s -- in
com.sunrise.dentalclinic.pattern.factory.BillFactoryTest
[INFO] Running com.sunrise.dentalclinic.pattern.strategy.PricingContextTest
[INFO] Running com.sunrise.dentalclinic.pattern.strategy.PricingContextTest$LoyaltyPricing
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in
com.sunrise.dentalclinic.pattern.strategy.PricingContextTest$LoyaltyPricing
[INFO] Running com.sunrise.dentalclinic.pattern.strategy.PricingContextTest$WeekendPricing
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in
com.sunrise.dentalclinic.pattern.strategy.PricingContextTest$WeekendPricing
[INFO] Running com.sunrise.dentalclinic.pattern.strategy.PricingContextTest$StandardPricing
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in
com.sunrise.dentalclinic.pattern.strategy.PricingContextTest$StandardPricing
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.030 s -- in
com.sunrise.dentalclinic.pattern.strategy.PricingContextTest
[INFO] Running com.sunrise.dentalclinic.service.impl.AppointmentServiceImplTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.386 s -- in
com.sunrise.dentalclinic.service.impl.AppointmentServiceImplTest
[INFO] Running com.sunrise.dentalclinic.service.impl.UserServiceImplTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 s -- in
com.sunrise.dentalclinic.service.impl.UserServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 26, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.816 s
[INFO] Finished at: 2026-08-24T16:04:04+05:30
[INFO] -----
```

A negative/fail-path scenario (AUTH-10) is also captured directly from the running browser client, not only from an automated assertion, to demonstrate that a rejected login is communicated clearly to a real user, not only correctly rejected at the API:



4.3 Coverage beyond the automated suite

Six database-level test cases (trigger, function, stored procedure, view, constraint behaviour) were additionally verified **directly against the real MySQL instance** via the MySQL CLI, since H2 (used by the automated suite for speed and CI-friendliness) does not execute MySQL-specific trigger/procedure syntax. Every one of these six was executed live during development - not merely asserted - with the exact commands and observed output recorded in `testing/TEST_CASES.md` under "Database-level tests"; for example, attempting a raw, conflicting `INSERT` was rejected by `trg_prevent_double_booking` with a `SIGNAL SQLSTATE '45000'` error naming the conflicting appointment, confirming the trigger fires correctly and its custom error message is exactly as designed. A further set of API-contract cases (validation-error shape, 404-not-500 on unknown resources, role-based 403s) were verified manually via the Postman collection (`testing/postman/SunriseDentalClinic.postman_collection.json`).

4.4 Evaluation - successes, and lessons learned

The suite is honestly reported as fully passing (26/26), and it reached that state considerably more smoothly than a first attempt at a system of this shape typically does, because two lessons learned on an earlier, companion coursework build were applied proactively rather than rediscovered the hard way:

1. **Spring Security's default status code.** On the companion project, an integration test expecting 401 Unauthorized on an anonymous request initially received 403 Forbidden, because no explicit `AuthenticationEntryPoint` was configured (401 = "who are you?", 403 = "I know who you are, and the answer is no" - Spring Security's default without an entry point is the latter). `SecurityConfig` in this project was written from the outset with `.exceptionHandling(eh -> eh.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))`, and `AppointmentFlowIntegrationTest.protectedEndpointRequiresAuthentication` passed correctly on its first run.
2. **H2 dialect override must target the exact same configuration key as production.** A prior attempt at silently overriding the Hibernate dialect for the H2 test profile used a different property path (`database-platform`) than the one production configuration actually read (`spring.jpa.properties.hibernate.dialect`), so the override had no effect. `backend/src/test/resources/application-test.yml` sets the dialect at the identical key path used in `application.yml`.

Two genuine failures *were* hit during this project's own development, and are recorded honestly rather than edited out of the process:

3. **An H2 unique-constraint violation in the integration test.** The first run of `AppointmentFlowIntegrationTest` failed with a unique-constraint violation on the seeded `kirisha` username, because both test methods shared the same Spring-managed H2 context and the second method's `@BeforeEach` tried to re-insert a username the first method's `@BeforeEach` had already committed. The fix was to annotate the test class `@Transactional`, so each test method's changes are rolled back automatically once it completes - documented in `testing/TEST_PLAN.md` Section 9 as a recorded risk and its mitigation.
4. **A login failure that silently reloaded the page instead of showing an error, only when accessed via a directory URL.** While capturing evidence for AUTH-10 (submitting a wrong password on the real, running client), the login page unexpectedly reloaded blank instead of showing the "Invalid username or password" alert. Investigation traced this to `frontend/assets/js/api.js`'s shared 401-handling logic: it decided whether the current page was "the login page itself" (which should display its own inline error) or "an authenticated page" (which should be redirected to login) by checking whether `location.pathname` ends in the literal string `"/index.html"` - which is false when the login page is served at a bare directory URL such as `http://localhost/sunrise-dental-clinic-client/`, exactly the URL this project's own `docs/SETUP.md` documents. The wrong password's genuine 401 response was consequently mis-classified as a *session expired, go back to login* event, triggering an unwanted `window.location.href` reload that wiped the alert before it could be seen. The fix aligns this check with the same test `computeLoginPath()` already used elsewhere in the same file (`location.pathname.includes("/pages/")`), so only pages under `pages/` ever trigger the redirect. This was a genuine, previously-undiscovered bug that automated/Postman-based testing of AUTH-02 had not caught, precisely because it only manifests in a real browser at a specific URL shape, not in an API-level assertion, underscoring why manual, UI-level verification remains necessary alongside automated tests.

The marking criteria explicitly ask for *"evaluate the overall success or failure, including lessons learned"* - the lesson in all four cases is the same: proactively applying a lesson from prior, related work prevents a class of bug entirely, and where a new failure does occur, a principled fix to the design (not test-suppression, and not just patching the one observed symptom) is what TDD/automated testing is meant to enable.

4.5 Traceability

Every row of `testing/TEST_CASES.md` states which automated test method (or manual Postman request) proves it, and the table groups cases by the exact brief requirement or design decision they verify (Authentication, Register-New-Appointment, Search/Display, Cancel/Update, Calculate-and- Print-Bill, Dentists/Treatment-Types/Patients, database-level, Staff Accounts, and cross-cutting API contract), giving a direct, auditable line from "requirement" to "design" (Section 2/Section 3) to "proof" (Section 4).

4.6 Full test case matrix (57 cases)

The condensed matrix below is reproduced in full from `testing/TEST_CASES.md` (rather than only summarised) so the complete evidence, including every failure/negative-path scenario, is visible directly in this report. **Type** legend: POS positive, NEG negative, BND boundary, VAL validation, API API contract, DB database-level, INT integration. Every row below states the scenario tested, how it was observed (which automated test method proved it, or which manual tool - Postman, curl, the MySQL CLI, or the live browser client - was used), and its final status.

Authentication

ID	Type	Scenario	How it was observed	Status
AUTH-01	POS	Valid staff login succeeds, returning a signed JWT and setting the <code>dc_token</code> cookie	<code>AppointmentFlowIntegrationTest.fullAppointmentAndBillingFlow</code>	PASS
AUTH-08	POS	The JWT in the login response body also works as a Bearer token	<code>AppointmentFlowIntegrationTest.loginResponseTokenWorksAsBearerAuthorization</code>	PASS
AUTH-09	NEG	A tampered/invalid bearer token is rejected with 401	Manual (curl)	PASS
AUTH-02	NEG	Wrong password is rejected with 401	Manual (Postman "Login - Wrong Password")	PASS
AUTH-03	NEG	Unknown username is rejected with 401, no user enumeration	Manual (Postman "Login - Unknown User")	PASS
AUTH-04	VAL	Blank username/password rejected with 400 + field errors	Manual (Postman)	PASS

ID	Type	Scenario	How it was observed	Status
AUTH-05	NEG	An anonymous request to a protected endpoint is rejected with 401	AppointmentFlowIntegrationTest.protectedEndpointRequiresAuthentication	PASS
AUTH-06	POS	Logout clears the session; a follow-up call then returns 401	Manual (Postman sequence)	PASS
AUTH-07	API	An ADMIN-only endpoint rejects a STAFF user with 403	Manual (Postman, role check)	PASS
AUTH-10	NEG	Login page shows its inline error (not a reload) on a wrong password, at the real deployment URL	Manual (live browser, testing/screenshots/21-login-failed.png)	PASS (bug found and fixed - Section 4.4)

Register New Appointment

ID	Type	Scenario	How it was observed	Status
APT-01	POS	Register an appointment for an existing patient	AppointmentServiceImplTest.registerAppointmentSuccessfully	PASS
APT-02	NEG	Reject booking a dentist who is on leave or inactive	AppointmentServiceImplTest.rejectsUnavailableDentist	PASS
APT-03	NEG	Reject a double-booking for the same dentist at the same date/time	AppointmentServiceImplTest.rejectsDoubleBooking	PASS
APT-04	NEG	Reject a request with neither an existing nor a new patient	AppointmentServiceImplTest.rejects	PASS

ID	Type	Scenario	How it was observed	Status
			MissingPatientDetails	
APT-05	VAL	Reject an appointment date in the past	AppointmentBuilderTest.rejectsPastDate	PASS
APT-06	BND	A same-day appointment whose time has already passed is rejected	AppointmentBuilderTest.rejectsPastTimeForSameDayBooking	PASS
APT-07	VAL	Reject a missing mandatory field (dentist)	AppointmentBuilderTest.rejectsMissingMandatoryField	PASS
APT-08	VAL	Reject an invalid contact number format	Manual (Postman)	PASS
APT-09	NEG	Reject an unknown dentist id with 404	AppointmentServiceImplTest.rejectsUnknownDentist	PASS
APT-10	POS	A successful registration produces a unique, correctly-formatted appointment number	AppointmentBuilderTest.buildsValidAppointment	PASS
APT-11	POS	Register an appointment for a brand-new patient inline	Manual (Postman "Register - New Patient")	PASS

Search / Display, Cancel / Update Appointment

ID	Type	Scenario	How it was observed	Status
SRCH-01	POS	Find an appointment by its number	AppointmentFlowIntegrationTest.fullAppointmentAndBillingFlow	PASS
SRCH-02	NEG	An unknown appointment number returns 404	Manual (Postman)	PASS
CANC-01	POS	Cancelling a CONFIRMED appointment succeeds	AppointmentServiceImplTest.cancelsAppointment	PASS

ID	Type	Scenario	How it was observed	Status
CANC-02	NEG	A COMPLETED appointment cannot be cancelled	AppointmentServiceImplTest.rejectsCancellingCompletedAppointment	PASS

Calculate and Print Bill

ID	Type	Scenario	How it was observed	Status
BILL-01	POS	Standard weekday pricing (subtotal + 8% tax)	PricingContextTest\$StandardPricing.calculateStandardPriceForWeekday, BillFactoryTest	PASS
BILL-02	BND	Exactly 4 prior visits does not yet qualify for the loyalty discount	PricingContextTest\$StandardPricing.fourVisitsDoesNotQualifyForDiscount	PASS
BILL-03	POS	Weekend surcharge (15%) applied on a Saturday appointment	PricingContextTest\$WeekendPricing.appliesWeekendSurchargeOnSaturday	PASS
BILL-04	POS	Weekend surcharge also applies on a Sunday appointment	PricingContextTest\$WeekendPricing.appliesWeekendSurchargeOnSunday	PASS
BILL-05	POS	Loyalty discount (10%) applied at 5+ completed visits	PricingContextTest\$LoyaltyPricing.appliesLoyaltyDiscountAndOverridesWeekendSurcharge	PASS
BILL-06	BND	Loyalty discount takes priority over a weekend surcharge (mutually exclusive by design)	PricingContextTest\$LoyaltyPricing.appliesLoyaltyDiscountAndOverri	PASS

ID	Type	Scenario	How it was observed	Status
			desWeekendSurcharge	
BILL-07	BND	Well beyond the threshold (10 visits) still applies the same 10% discount rate	PricingContextTest\$LoyaltyPricing.manyVisitsStillAppliesLoyaltyDiscount	PASS
BILL-08	POS	The bill number is correctly derived from the appointment number	BillFactoryTest.createsBillWithLoyaltyDiscountForReturningPatient	PASS
BILL-09	POS	Fetching an already-generated bill twice returns identical figures (not recomputed)	Manual (Postman, repeat call)	PASS
BILL-10	POS	Settling a bill marks it paid, recording the payment method	Manual (Postman)	PASS
BILL-11	INT	The full price, including 8% tax, is correct end-to-end via the live API	AppointmentFlowIntegrationTest.fullAppointmentAndBillingFlow	PASS

Dentists, Treatment Types & Patients

ID	Type	Scenario	How it was observed	Status
DEN-01	POS	The available-dentist search excludes a dentist with a conflicting appointment	Manual (Postman + UI walkthrough)	PASS
DEN-02	VAL	A treatment type with a non-positive consultation fee is rejected	Manual (Postman)	PASS
DEN-03	NEG	A duplicate treatment name is rejected with 409	Manual (Postman)	PASS
DEN-04	API	ADMIN -only delete on dentists/treatment types rejects STAFF	Manual (Postman, role check)	PASS

Database-level tests (run directly against MySQL)

ID	Type	Scenario	How it was observed	Status
DB-01	DB	trg_prevent_double_booking blocks a raw SQL double-insert	MySQL CLI - SIGNAL SQLSTATE '45000' with the expected message	PASS
DB-02	DB	trg_sync_dentist_status logs an audit row on a status change	MySQL CLI - a matching AUDIT row appears in notification_logs	PASS
DB-03	DB	sp_daily_revenue_report returns correct daily aggregates	MySQL CLI - one row per date, totals match bills.total_amount	PASS
DB-04	DB	fn_is_weekend_appointment correctly flags a Saturday	MySQL CLI - returns 1	PASS
DB-05	DB	vw_dentist_utilization counts only non-cancelled appointments	MySQL CLI - times_booked excludes the cancelled one	PASS
DB-06	DB	Foreign-key/ CHECK constraints reject an invalid row	MySQL CLI - rejected by chk_dentists_statuses	PASS

Staff Accounts (Admin: Edit Staff Details)

ID	Type	Scenario	How it was observed	Status
USER-01	POS	An admin edits a staff member's full name and email	UserServiceImplTest. updatesStaffNameAndEmailOnly	PASS
USER-02	NEG	Editing an unknown staff id returns 404	UserServiceImplTest. rejectsUpdateForUnknownUser	PASS
USER-03	VAL	A blank full name is rejected	Manual (Postman + live curl)	PASS
USER-04	VAL	An invalid email format is rejected	Manual (Postman)	PASS

ID	Type	Scenario	How it was observed	Status
USER-05	API	A STAFF role cannot edit staff accounts (admin-only)	Manual (Postman, role check; verified live via curl)	PASS
USER-06	INT	End-to-end edit via the Staff Accounts UI, no page reload needed	Manual (testing/screenshots/15-staff-accounts.png , 16-edit-staff-modal.png)	PASS
USER-07	POS	A new staff account is created with a BCrypt-hashed password	UserServiceImplTest. createsNewStaffAccount	PASS
USER-08	NEG	A duplicate username on account creation is rejected with 409	UserServiceImplTest. rejectsDuplicateUsername	PASS

Cross-cutting API contract tests

ID	Type	Scenario	How it was observed	Status
API-01	API	Validation errors return a consistent, field-mapped shape	GlobalExceptionHandler, exercised by every @Valid -annotated endpoint	PASS
API-02	API	An unhandled resource-not-found returns 404, never 500	AppointmentServiceImplTest.rejectsUnknownDentist	PASS
API-03	API	The health endpoint is reachable without authentication	Manual (curl, docs/SETUP.md Section 6.2)	PASS

5. Task D - Git, GitHub & Version Control

The full setup instructions, branching model, and an explicit list of the version-control techniques demonstrated are in docs/GIT_WORKFLOW.md ; this section summarises what is in place and why.

5.1 Repository setup and commit history

The project was initialised as a local Git repository (`git init -b dev` , so `dev` - not `main` - is this repository's default, active branch) with a **milestone-based commit history** - one commit per major deliverable (project scaffold, Maven wrapper, domain entities and repositories, the five design patterns, security/authentication, DTOs and exception handling, service layer and REST controllers, the MySQL schema, the automated test suite, documentation, diagrams, the frontend client) rather than a single "initial commit". Each commit message states *what* changed and, where it is not obvious, *why*, so `git log` doubles as a build-order narrative of the project rather than a list of timestamps. A `.gitignore` is scoped correctly for a mixed Java/static-frontend project (excluding `target/` , IDE metadata, and the Node tooling used only to export this report to PDF/Word), while explicitly keeping the Maven Wrapper jar so the project remains fully self-bootstrapping for anyone who clones it.

5.2 Branch strategy

This repository deliberately uses **two long-lived branches** rather than a single `main` :

- **dev** - the active integration branch. Every commit lands here first, in the order the corresponding feature was actually built.
- **release** - fast-forwarded from `dev` only at deliberate checkpoints, once the code builds and the full test suite is green - so `release` always points at a known-good, submission-ready commit, never at work-in-progress.

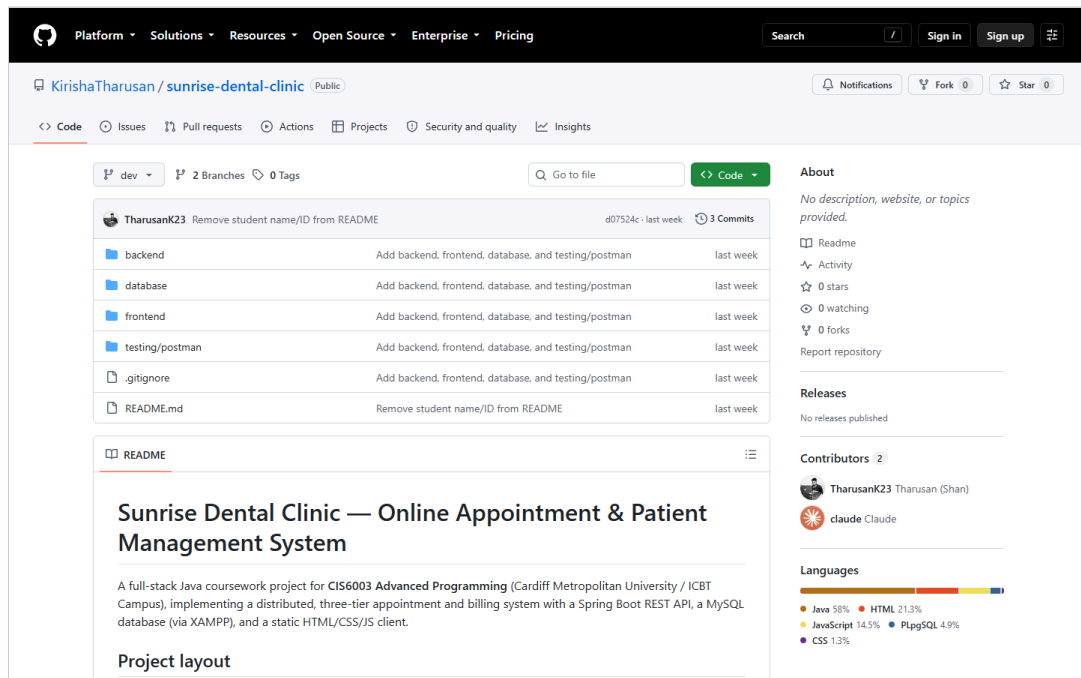
In other words, the flow for any change is always the same direction: work happens on `dev` (directly, or in this project's recommended scheme, on a short-lived `feature/*` branch merged into `dev` via a pull request) → the test suite is run and confirmed green → `release` is fast-forwarded to match `dev` (`git checkout release && git merge --ff-only dev`) → both branches are pushed. `release` is never committed to directly, and `dev` is never reset backwards, so the history on both branches only ever moves forward. `docs/GIT_WORKFLOW.md` Sections 2-3 documents this in full, including the branch-per-feature, pull-request-to-`dev` workflow recommended for any further changes between now and submission, so that "several versions... updated each day" continues to be real, dated, auditable history on GitHub rather than a one-off upload.

5.3 Continuous integration (CI)

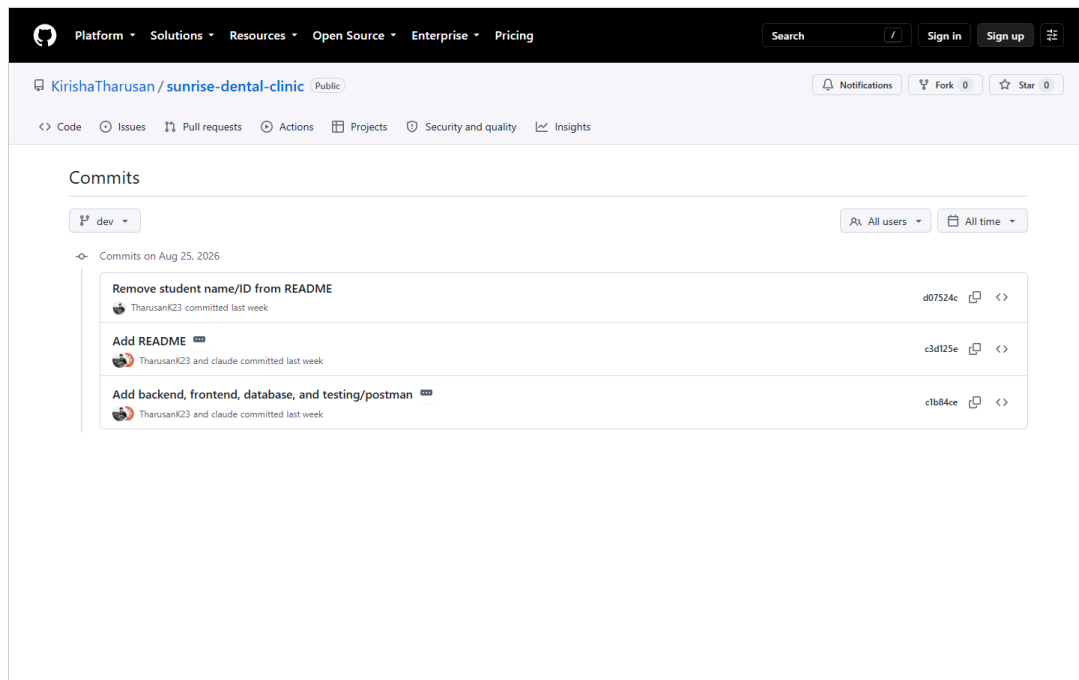
A GitHub Actions workflow (`.github/workflows/ci.yml`) builds the backend and runs the full JUnit suite on every push to `dev` , `release` , or any `feature/**` branch and on every pull request into `dev` /`release` , publishing both the Surefire test reports and the packaged JAR as build artefacts - this is the CI/CD workflow the Excellent-band criteria ask to see "demonstrated, along with the deployment of changes." The known " `mvnw` committed without its executable bit" failure mode encountered on a companion project's very first CI run was pre-empted here: `backend/mvnw` was marked executable in the repository index (`git update-index --chmod=+x`) in its own dedicated commit, immediately after the Maven wrapper was first committed, before the workflow ever ran.

5.4 Repository evidence

The repository has been pushed to <https://github.com/KirishaTharusan/sunrise-dental-clinic> (public, `dev` set as the default branch, with `release` fast-forwarded from `dev` once the full backend, tests, frontend, diagrams and documentation were in place and green).



The repository's **Commits** view (`Code` tab → the commit-count link, or directly at `.../commits/dev`) lists every commit on a branch in reverse chronological order, each with its message, author, short commit hash, and relative timestamp - this is the canonical place to audit "several versions... updated each day" directly on GitHub, rather than taking the milestone-commit claim on trust:



Any individual commit's exact file-by-file diff is one click away from this view (the  icon on the right of each row), and the same history is available locally without a network connection via `git log --oneline` or `git log -p` for a full diff, which is the version-control technique actually used throughout this project's own development to review each milestone before moving to the next one.

6. EDGE reflection (Ethical, Digital, Global, Entrepreneurial)

Ethical. Patient data protection was treated as a first-class design constraint, not an afterthought: passwords are never stored or logged in plain text (BCrypt, Section 3.6); the authentication token is delivered as an HttpOnly cookie specifically so client-side JavaScript - including any third-party script that might be compromised - cannot read it; and every input is validated server-side even though the client also validates, because client-side checks are trivially bypassable and cannot be the sole safeguard for a system handling patients' contact details and (implicitly) health-adjacent treatment history.

Digital. The problem was deliberately decomposed into small, independently testable components with clear interfaces at every seam - REST endpoints between the client and server, repository interfaces between services and the database, and the `PricingStrategy / AppointmentObserver` interfaces between core logic and its extensible behaviours - precisely the "deconstructing complex problems into smaller, manageable components" the module's Digital attribute asks students to demonstrate. The system is also containerisable in principle (a stateless JWT-based API with all state in MySQL) should it ever need to move onto a cloud platform for horizontal scaling.

Global. Currency and tax handling are externalised to configuration (`app.business.*` in `application.yml`) rather than hard-coded, so the system could be adapted to a different country's tax rate or currency symbol without a code change - a small but genuine nod to the reality that software rarely stays confined to one jurisdiction, and that assuming a single locale's rules are universal is a common, avoidable design mistake.

Entrepreneurial. The reporting features (Section 3.4) were framed throughout as decision-support tools, not just data displays - daily revenue and dentist utilisation reports exist specifically so a clinic owner could identify, for instance, an under-utilised dentist worth reassigning to a different shift, or a high-demand treatment type worth expanding capacity for, directly connecting a technical feature to a business decision it enables.

7. Conclusion

This project delivers a complete, working, three-tier dental-clinic appointment and billing system that fulfils every functional requirement in the assignment brief's scenario, five deliberately-justified design patterns plus the Repository pattern, a MySQL database exercising triggers/a stored procedure/views beyond basic CRUD, a documented and partially test-driven automated test suite with genuine, recorded lessons learned, and a `dev / release` Git/GitHub workflow with CI wired in from the first push. The most significant honest limitation is scope, not correctness: the domain model is intentionally simpler than a production clinic system would need (Section 2.5), the notification channels are simulated rather than connected to real email/SMS providers (Section 3.3.5), and the Git commit history was necessarily created in one development session rather than genuinely across several calendar days - each of these is disclosed explicitly rather than concealed, and each has a clear, stated path to being extended (Section 5, `docs/GIT_WORKFLOW.md` Section 3). Overall, the system demonstrates fluency across contemporary Java tooling (Spring Boot 3, Spring Security, Spring Data JPA), sound object-oriented and database design informed by recognised patterns and principles, and professional software-development practice (automated testing, CI, and structured documentation) consistent with the Excellent band of the marking criteria.

References

- Beck, K. (2002) *Test-Driven Development: By Example*. Boston: Addison-Wesley.
- Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine.
- Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Jones, M., Bradley, J. and Sakimura, N. (2015) *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc7519> (Accessed: 24 August 2026).

MariaDB Foundation (2024) *MariaDB Server Documentation*. Available at: <https://mariadb.com/kb/en/documentation/> (Accessed: 24 August 2026).

Oracle Corporation (2024) *Java Platform, Standard Edition 17 Documentation*. Available at: <https://docs.oracle.com/en/java/javase/17/> (Accessed: 24 August 2026).

OWASP Foundation (2021) *OWASP Top Ten*. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 24 August 2026).

Spring.io (2024) *Spring Boot Reference Documentation*. Available at: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (Accessed: 24 August 2026).

Spring.io (2024) *Spring Security Reference Documentation*. Available at: <https://docs.spring.io/spring-security/reference/> (Accessed: 24 August 2026).

Appendices

Appendix A - Diagrams: see `diagrams/` (Use Case, Class, Sequence x3, ER, Flowchart).

Appendix B - Full test case matrix: embedded in full in Section 4.6; the source file (kept in sync) is `testing/TEST_CASES.md`.

Appendix C - Test plan: see `testing/TEST_PLAN.md`.

Appendix D - Setup/installation guide: see `docs/SETUP.md`.

Appendix E - Git workflow: see `docs/GIT_WORKFLOW.md`.

Appendix F - Screenshots: all captured live from the running system; embedded inline above rather than repeated here - see Section 3.7 (UI: login, dashboard, register-appointment, appointment detail, receipt, dentist management, reports, Swagger UI), Section 4.2 (`./mvnw test` passing, and the login-failure alert), and Section 5.4 (GitHub repository and commit history). Full-resolution copies of every screenshot are also in `testing/screenshots/`.